

Projektspezifische Notizen

Alain Horner · 16. September 2026

Dieses Dokument fasst die projektspezifische Übergabe von [Alain](#) zusammen. Es ergänzt [AI-Setup & Arbeit mit Agenten](#).

Was gemacht wurde

Eine chronologische Übersicht der geleisteten Arbeit findet sich in den [Status-Updates Alain](#) im Confluence. Dort sind jeweils auch besprochene **"Inputs Entwicklungsprozess"** erfasst: Beobachtungen und Verbesserungsvorschläge zum Entwicklungsprozess, die über den reinen Code hinausgehen.

Etablierte Patterns (Exam/ExamTemplate)

Die folgenden Patterns wurden im Bereich Exam/ExamTemplate angewendet. Sie sollten nach Möglichkeit auch im übrigen bestehenden Code reflektiert werden, sobald dieser angefasst wird (continuous improvement).

Code-Organisation

- **Business-Logik** gehört ins **Backend**, wo immer möglich.
- **Komponenten klein und wiederverwendbar halten**. Lieber mehrere fokussierte Komponenten als eine grosse.
- **Existierende Komponenten wiederverwenden**, bevor Neues gebaut wird (z.B. FormBuilder, `BasicGrid.vue`, ...).
- **Reaktive Logik in Composables (Hooks)** auslagern, statische Logik in Helpers.
- **Ablageort: "as local as possible, but as global as necessary"**: Gilt für Komponenten, Helpers und Composables gleichermassen.
- **TODOs immer mit Namen** (z.B. `// TODO @andrei: replace this by TanStackQuery`), damit die Verantwortlichkeit klar ist.

Daten

- **Data-Fetching via TanStack Query und HeyAPI** über `apiService.ts`. Kein manuelles Fetching in Komponenten.
- **Keine Umformung der Requests im Frontend-Proxy**: Der fe-server leitet nur weiter; jede Transformation gehört entweder ins Backend oder in den Client.

State

- **Möglichst wenige Stores.** Ausnahmen sind gerechtfertigt, wo State über viele Schritte geteilt wird (z.B. ein grösserer Wizard). Ansonsten gilt:
 - Temporärer State: `computed` / `ref` in der Komponente oder im Composable.
 - Teilbarer State: In die URL (Deep-Links, Browser-Navigation ist gratis abgedeckt).
 - Permanenter State: Ins Backend.

Styling

- **Generelles Styling über das Vuetify-Theme** statt Inline-Styles.
- **Abweichungen vom Theme** in den betroffenen Komponenten über Vuetify-Utility-Classes lösen; nicht über Inline-Styles.

Empfehlungen für weitere Schritte (priorisiert)

Die folgenden Empfehlungen sind meine persönliche Einschätzung aus der Projektarbeit. Sie sind nicht als Pflichtenheft gedacht, sondern als priorisierte Diskussionsgrundlage.

1. **Frontend-Tests mit gemocktem Backend aufbauen.** Playwright-Tests mit gemockten API-Responses (Route-Mocking). Diese können sehr schnell durchlaufen, auch ohne Backend und DB (lokal oder auch auf jedem PR). Dies ist der wichtigste deterministische Feedback-Loop für Agenten und erlaubt schnelle Iteration.

Wichtig dabei: Damit der Feedback-Loop funktioniert, sollten die Tests im gleichen Repository liegen und idealerweise zusammen mit dem entsprechenden Feature geschrieben werden. Sonst driften sie vom Code weg und werden für Agenten unsichtbar.

Abgrenzung:

- Das korrekte Funktionieren der API (inkl. OpenAPI-Spec) sollte mit dedizierten Tests im Backend-Repository getestet werden.
- Echte E2E-Tests (vom Browser bis zur DB) haben daneben weiterhin ihren Platz, und können z.B. einmal pro Nacht ausgeführt werden. Ich würde diese eher als "smoke tests" sehen, welche die korrekte Verknüpfung der einzelnen Teile (Frontend, Proxy-Server, Backend, DB) überprüfen und nicht jeden möglichen Pfad abdecken. Für schnelles Iterieren und die Arbeit von Agenten sind solche Tests zu langsam.

2. **Lücken in der OpenAPI-Spec schliessen.** Die generierten Types (`client/src/api/seb-server/generated/hey-api/types.gen.ts`) enthalten aktuell noch viele `unknown` Types (`unknown` bedeutet "kein TypeCheck und somit kein Schutz"). Zudem sind meines Wissens noch nicht alle APIs auf HeyAPI migriert.

3. **Vendor-Updates institutionalisieren.** Ein `.github/dependabot.yml` -File erstellen und einchecken. Dieses kann so konfiguriert werden, dass z.B. monatlich automatisch ein PR mit Vendor-Updates erstellt wird. Die bestehende PR-Pipeline (Build, Format, Tests, TypeCheck) wird darauf angewendet; die PRs müssen dann nur noch reviewed und gemerged werden.

4. Supply-Chain-Sicherheit pragmatisch weiterführen. Das exakte Versions-Pinning und der [Shai-Hulud-Scan](#) im Deploy-Workflow (`.github/workflows/auto-test-deploy.yml`) sollten unbedingt beibehalten werden. Mit einem Wechsel zu pnpm könnte man zudem [minimumReleaseAge](#) einschalten. [OWASP NPM Security Cheat Sheet](#) hat zudem weitere hilfreiche Sicherheitsempfehlungen.

5. Workspaces für `client` + `server` einrichten. (npm oder pnpm) Der fe-server hat momentan weder Prettier noch ESLint, TypeCheck-Gate oder Precommit-Hook. Mit einem Workspace-Setup können sich beide Teile die Konfiguration teilen.

6. Weitere deterministische Feedback-Loops evaluieren. Dies steigert die Output-Qualität von Agenten.

Beispiele:

- `strictTemplates` in `client/tsconfig.json`: TypeChecking innerhalb der Vue-Templates. Aktuell ausgeschaltet wegen eines bekannten [Vuetify-Issues](#). Möglicherweise gibt es einen Workaround. Spätestens bei einem Vuetify-Major-Update neu prüfen.
- Die ESLint-[complexity](#)-Rule als strikte Obergrenze für die Komplexität einer einzelnen Funktion.

7. `moment-timezone` durch native `Intl` -APIs ersetzen. Dieser Vendor ist der grösste Brocken in unserem Bundle und wird an nur wenigen Stellen verwendet. Das Paket sollte durch native Browser API calls (`Intl.supportedValuesOf("timeZone")`) ersetzt werden können.

8. Accessibility-Checks im Linting. [eslint-plugin-vuejs-accessibility](#) in die bestehende ESLint-Konfiguration (`client/eslint.config.js`) aufnehmen. Einmal eingerichtet, läuft der Check gratis in jedem Lint-Lauf und damit in jedem PR mit.

9. Lighthouse-Audit. Ein [Lighthouse](#)-Audit für Performance, Accessibility und Best Practices. Für eine interne Applikation nicht zwingend, aber evtl. können damit einige "low hanging fruits" gefunden werden.

10. Vite+ evaluieren. [Vite+](#) verspricht, mittelfristig Test-Runner, Linting, Bundling und Monorepo-Tasks in einer einzigen, deutlich schnelleren Toolchain zu konsolidieren. Vite+ sollte im Auge behalten und bei Gelegenheit evaluiert werden.